OS & SP

Name : D.Teja Sadwika **Roll No :** 2520030536

SKILL – 21

1.Skill21.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#include <errno.h>

#define MAX_INPUT 256
#define MAX_ARGS 64

/* Display error message */
void print_error(const char *message)
{
```

```
    fprintf(stderr, "ERROR: %s\n", message);
}

/* Parse command */
int parse_command(char *input, char *args[])
{
    int argc = 0;
    char *token = strtok(input, " ");

    while (token != NULL && argc < MAX_ARGS - 1) {
        args[argc++] = token;
        token = strtok(NULL, " ");
    }

    args[argc] = NULL;
    return argc;
}
```

```
/* Execute command */
int execute_command(char *args[])
{
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        return -1;
    }

    /* Child process */
    if (pid == 0) {
        execvp(args[0], args);

        /* Executes only if execvp fails */
        fprintf(stderr,
                "ERROR: Cannot execute '%s': %s\n",
                args[0], strerror(errno));
```

```
    exit(127);
}

/* Parent process */
else {
    int status;

    if (waitpid(pid, &status, 0) < 0) {
        perror("waitpid");
        return -1;
    }

    if (WIFEXITED(status)) {
        int exit_status = WEXITSTATUS(status);

        if (exit_status != 0) {
            printf("Command exited with status %d\n",
```

```
                exit_status);
        }

        return exit_status;
    }

    if (WIFSIGNALED(status)) {
        printf("Command terminated by signal %d\n",
            WTERMSIG(status));
        return -1;
    }
  }

  return 0;
}

int main()
{
```

```
char input[MAX_INPUT];

printf("====================================\n");
printf(" OSSP PROGRAM 21\n");
printf(" Integration and Error Handling\n");
printf("====================================\n");

printf("Type Linux commands.\n");
printf("Type 'help' for available commands.\n");
printf("Type 'exit' to quit.\n\n");

while (1) {
    printf("shell> ");
    fflush(stdout);

    /* Read user input */
    if (fgets(input, sizeof(input), stdin) == NULL) {
        printf("\nInput error or EOF.\n");
```

```
    break;
}

/* Remove newline */
input[strcspn(input, "\n")] = '\0';

/* Empty command */
if (strlen(input) == 0) {
    printf("ERROR: Empty command.\n");
    continue;
}

/* Exit */
if (strcmp(input, "exit") == 0) {
    printf("Exiting safely...\n");
    break;
}
```

```
/* Help */
if (strcmp(input, "help") == 0) {
    printf("\nAvailable commands:\n");
    printf(" help - display help\n");
    printf(" exit - exit shell\n");
    printf(" Any Linux command\n\n");
    continue;
}

/* Copy input because strtok modifies it */
char command_copy[MAX_INPUT];
strcpy(command_copy, input);

/* Argument array */
char *args[MAX_ARGS];

/* Parse command */
int argc = parse_command(command_copy, args);
```

```
    /* Validate parser output */
    if (argc == 0) {
        print_error("Invalid command syntax.");
        continue;
    }

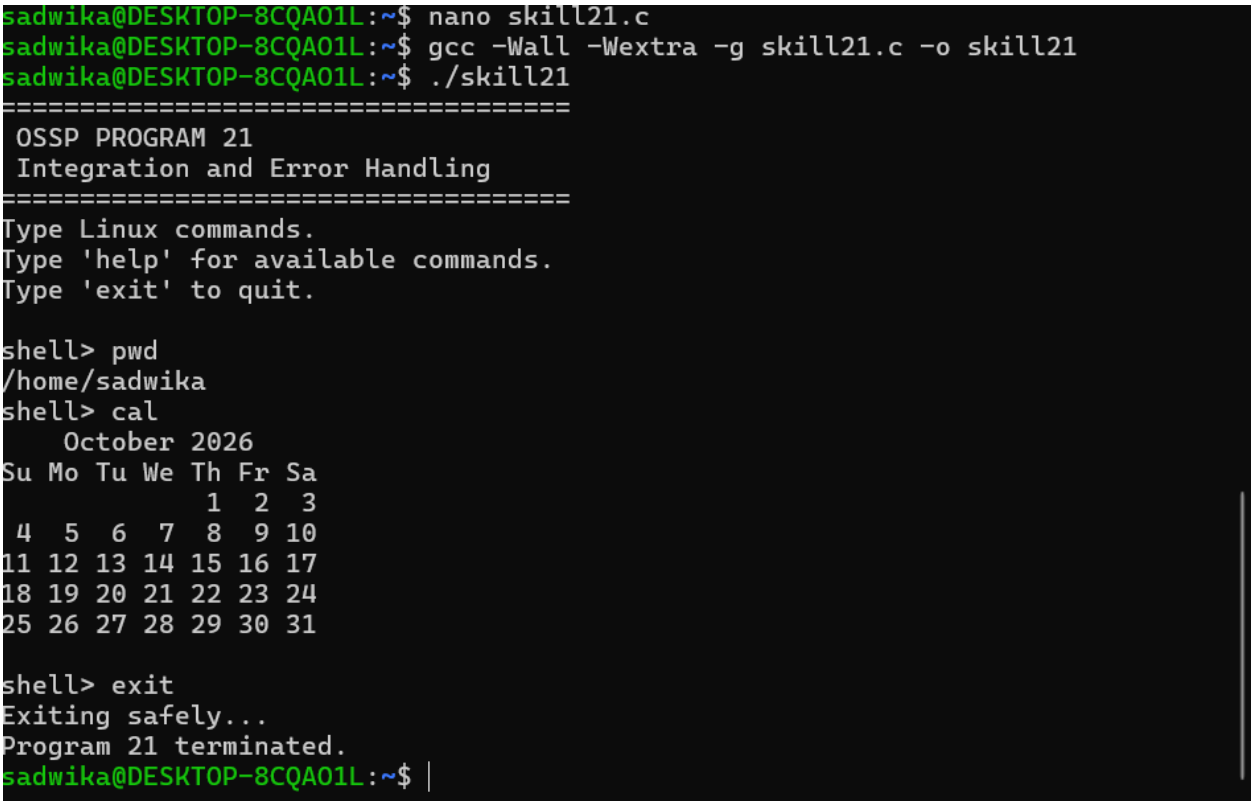
    /* Execute command */
    int result = execute_command(args);

    /* Runtime error handling */
    if (result == -1) {
        print_error("Command execution failed.");
    }
}

printf("Program 21 terminated.\n");
return 0;
```

}

OUTPUT :


sadwika@DESKTOP-8CQAO1L:~$ nano skill21.c
sadwika@DESKTOP-8CQAO1L:~$ gcc -Wall -Wextra -g skill21.c -o skill21
sadwika@DESKTOP-8CQAO1L:~$ ./skill21
OSSP PROGRAM 21
Integration and Error Handling
Type Linux commands.
Type 'help' for available commands.
Type 'exit' to quit.
shell> pwd
/home/sadwika
shell> cal
October 2026
Su Mo Tu We Th Fr Sa
1 2 3
4 5 6 7 8 9 10
11 12 13 14 15 16 17
18 19 20 21 22 23 24
25 26 27 28 29 30 31
shell> exit
Exiting safely...
Program 21 terminated.
sadwika@DESKTOP-8CQAO1L:~$